Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №6

«Автоматизоване тестування Python-програм»

з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФеП-32

Братівник Анастасія

Викладач: доц. Куньо І. М.

Львів – 2026

Мета: Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest: проєктування unit та integration tests, створення fixtures, використання параметризації, перевірка exceptions, ізоляція файлової системи та зовнішніх залежностей через tmp_path, monkeypatch, Mock, AsyncMock, а також аналіз test coverage та branch coverage. https://github.com/BtativnykA/Brativnyk_Anastasiia_Lab06

Індивідуальний варіант і постановка задачі

Варіант №1 — «Система обліку студентів».

Протестовано:

- створення Student та валідацію обов'язкових полів;
- Grade як immutable value object із boundary-перевірками [0, 100];
- пошук студента за прізвищем;
- фільтрацію студентів за групою;
- обчислення середнього бала групи;
- визначення найкращого студента;
- порожню колекцію студентів;
- CSV/JSON export;
- YAML/env конфігурацію;
- StudentRepository через real in-memory implementation;
- Notifier, Exporter, GradePolicy через Mock із return_value, side_effect та interaction assertions;
- async external client через AsyncMock;
- integration flow service → repository → exporter.

Test suite містить **52 тести** (unit + integration), запускається однією командою та формує coverage report.

Структура

Структура production code

src/student_system/

├── __init__.py

├── exceptions.py — власна ієрархія винятків

├── value_objects.py — Grade (frozen dataclass)

├── models.py — Student (domain entity)

├── repositories.py — StudentRepository (ABC) + InMemoryStudentRepository

├── protocols.py — Notifier, Exporter, GradePolicy (Protocol)

├── services.py — StudentService (orchestration)

├── exporters.py — JsonExporter + export_students_to_file

├── config.py — RuntimeConfig (env vars)

└── async_service.py — AsyncStudentClient

Структура test suite

tests/

├── __init__.py

├── conftest.py — shared fixtures

├── unit/

│ ├── __init__.py

│ ├── test_value_objects.py — 11 tests (Grade)

│ ├── test_models.py — 7 tests (Student)

│ ├── test_services.py — 17 tests (StudentService + Mock)

│ ├── test_exporters.py — 5 tests (JsonExporter + tmp_path)

│ ├── test_config.py — 3 tests (monkeypatch)

│ └── test_async_service.py — 2 tests (AsyncMock)

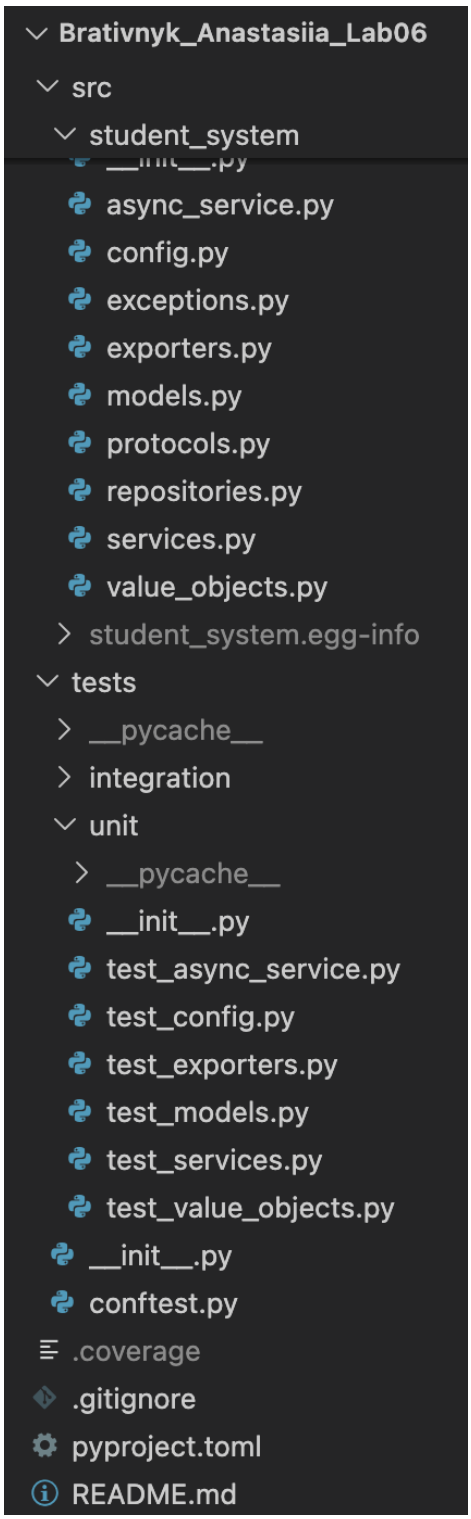
└── integration/

    ├── __init__.py

    ├── test_import_export.py   — 2 tests (export → файл → JSON)

    └── test_full_flow.py   — 2 tests (service + repo + exporter + notifier)

Разом: 52 тести (49 unit + 3 integration з params).



Test matrix

Компонент	Normal	Boundary	Invalid input	Failure path
Grade	90.0	0, 100	-0.01, 100.01	ValueError
Student	creation	ID=1	ID=0, empty names	ValidationError
InMemoryStudentRepository	add/get	empty	duplicate	DuplicateError, NotFoundError
StudentService	register, find, filter	empty repo	duplicate	Notifier side_effect → ExternalServiceError
JsonExporter	1, many	empty	—	—
export_students_to_file	tmp_path	empty	—	—
load_config	defaults	partial env	—	—
AsyncStudentClient	success	—	external exception	ExternalServiceError

Приклади unit tests

Unit test для Grade

```python
def test_grade_creation() -> None:
    g = Grade(90.0)
    assert g.value == 90.0


def test_negative_grade_rejected() -> None:
    with pytest.raises(ValueError, match="Grade must be in"):
        Grade(-1.0)


def test_grade_average_with_add() -> None:
    result = Grade(80.0) + Grade(90.0)
```

```
    assert result.value == pytest.approx(85.0)
```

Unit test для Student (validation)

```
def test_student_invalid_id() -> None:
    with pytest.raises(ValidationError, match="student_id"):
        Student(0, "A", "B", "PI-21", Grade(80.0))


def test_student_empty_first_name() -> None:
    with pytest.raises(ValidationError, match="first_name"):
        Student(1, "  ", "B", "PI-21", Grade(80.0))
```

Unit test для StudentService з Mock

```
def test_register_sends_notification(service: StudentService) -> None:
    new_student = Student(10, "Petro", "Novak", "PI-23", Grade(88.0))
    service.register(new_student)
    notifier = service._notifier
    notifier.send.assert_called_once_with(
        "registry@university",
        "Registered: Petro Novak",
    )
```

Приклад integration test

```
@pytest.mark.integration
def test_full_student_lifecycle() -> None:
    repo = InMemoryStudentRepository()
    notifier = Mock(spec=Notifier)
    exporter = Mock(spec=Exporter)
    exporter.export.return_value = "EXPORTED"
    policy = Mock(spec=GradePolicy)
    policy.apply.side_effect = lambda g: g

    service = StudentService(repo, notifier, exporter, policy)

    # register
    s = Student(1, "Anna", "Koval", "PI-21", Grade(91.5))
    service.register(s)
    assert len(repo) == 1
```

```
    notifier.send.assert_called_once()

    # update
    updated = service.update_grade(1, Grade(95.0))
    assert updated.grade.value == pytest.approx(95.0)

    # queries
    assert service.best_student() is updated
    assert service.average_of_group("PI-21") == pytest.approx(95.0)

    # export
    service.export_all()
    exporter.export.assert_called_once()
```

Чим integration test відрізняється від unit:

- Тестує **кілька компонентів разом** (service + repository + protocols).
- Перевіряє **реальну взаємодію** (не тільки ізольований метод).
- Використовує реальний InMemoryStudentRepository замість Mock.

Приклад fixture

tests/conftest.py:

```
@pytest.fixture
def student() -> Student:
    """A single demo student."""
    return Student(
        student_id=1,
        first_name="Anna",
        last_name="Koval",
        group="PI-21",
        grade=Grade(91.5),
    )


@pytest.fixture
def students() -> list[Student]:
    """Three demo students in different groups."""
    return [
        Student(1, "Anna", "Koval", "PI-21", Grade(91.5)),
        Student(2, "Oleh", "Melnyk", "PI-21", Grade(84.0)),
```

```
        Student(3, "Iryna", "Bondar", "PI-22", Grade(95.2)),
    ]
```

Використання у тесті:

```
def test_student_creation(student: Student) -> None:
    assert student.student_id == 1
    assert student.full_name == "Anna Koval"
```

Fixture із yield (setup + teardown)

```
@pytest.fixture
def repository(students: list[Student]) -> Iterator[InMemoryStudentRepository
]:
    """Pre-populated repository (fixture with yield for cleanup)."""
    repo = InMemoryStudentRepository()
    for s in students:
        repo.add(s)
    yield repo
    # teardown — нічого чистити, бо in-memory
```

Усе, що **до** yield — setup; усе, що **після** — teardown.

Приклад parametrization

Boundary-параметризація для Grade

```
@pytest.mark.parametrize(
    ("raw", "expected"),
    [
        pytest.param(0.0, 0.0, id="minimum"),
        pytest.param(50.0, 50.0, id="middle"),
        pytest.param(99.99, 99.99, id="near-max"),
        pytest.param(100.0, 100.0, id="maximum"),
    ],
)
def test_valid_grade_boundaries(raw: float, expected: float) -> None:
    assert Grade(raw).value == pytest.approx(expected)


@pytest.mark.parametrize("raw", [-0.01, 100.01, -100.0, 1000.0])
def test_invalid_grade_boundaries(raw: float) -> None:
    with pytest.raises(ValueError):
        Grade(raw)
```

Параметризація для фільтрації за групою

```
@pytest.mark.parametrize(
    ("group", "expected"),
    [
        ("PI-21", 2),
        ("PI-22", 1),
        ("PI-99", 0),
    ],
)
def test_group_counts(service: StudentService, group: str, expected: int) ->
None:
    assert len(service.filter_by_group(group)) == expected
```

Boundary test

Для Grade (діапазон 0–100) перевірено **7 значень**:

Значення	Очікування
-0.01	помилка
0.0	valid
0.01	valid
50.0	valid
99.99	valid
100.0	valid
100.01	помилка

Це значно сильніше, ніж один тест зі значенням 50.

pytest.raises

```
def test_negative_grade_rejected() -> None:
    with pytest.raises(ValueError, match="Grade must be in"):
        Grade(-1.0)
```

Перевірка **message** через match= — регулярний вираз.

Перевірка **кастомного exception з деталями**:

```
def test_student_invalid_id() -> None:
    with pytest.raises(ValidationError, match="student_id"):
```

```
        Student(0, "A", "B", "PI-21", Grade(80.0))
```

pytest.approx

```
def test_grade_average_with_add() -> None:
    result = Grade(80.0) + Grade(90.0)
    assert result.value == pytest.approx(85.0)
```

Float-порівняння через == небезпечне через представлення IEEE 754. pytest.approx дозволяє припустиму похибку.

tmp_path

```
def test_export_to_file(tmp_path, students: list[Student]) -> None:
    output = tmp_path / "students.json"
    count = export_students_to_file(students, output)
    assert count == 3
    assert output.exists()
    data = json.loads(output.read_text(encoding="utf-8"))
    assert len(data) == 3
```

tmp_path — це унікальна тимчасова директорія для кожного тесту. **Ізоляція** файлової системи, без garbage у репозиторії.

monkeypatch

```
def test_config_defaults(monkeypatch) -> None:
    monkeypatch.delenv("APP_ENV", raising=False)
    monkeypatch.delenv("PAYMENT_MODE", raising=False)
    monkeypatch.delenv("DEFAULT_GROUP", raising=False)
    config = load_config()
    assert config.environment == "development"
    assert config.payment_mode == "sandbox"


def test_config_from_env(monkeypatch) -> None:
    monkeypatch.setenv("APP_ENV", "testing")
    config = load_config()
    assert config.environment == "testing"
```

Після тесту змінні **автоматично відкочуються** — тест не впливає на інші.

Mock

```
from unittest.mock import Mock
```

```
from student_system.protocols import Notifier

notifier = Mock(spec=Notifier)
```

spec=Notifier обмежує API — якщо у тесті викликати метод, якого немає в Protocol, отримаємо AttributeError. Це **запобігає over-mocking**.

return_value

```
exporter = Mock(spec=Exporter)
exporter.export.return_value = "[]"
```

side_effect — failure path

```
notifier = Mock(spec=Notifier)
notifier.send.side_effect = RuntimeError("network down")

service = StudentService(repository, notifier, exporter, policy)
new_student = Student(20, "Taras", "Shevchenko", "PI-23", Grade(90.0))
with pytest.raises(ExternalServiceError, match="Notifier failed"):
    service.register(new_student)
```

Перевірено: **production code обгортає сторонній RuntimeError у domain-specific ExternalServiceError**.

Interaction assertion

```
notifier.send.assert_called_once_with(
    "registry@university",
    "Registered: Petro Novak",
)
```

Перевіряє **не тільки те, що метод викликано**, а й **з якими саме аргументами** і **скільки разів**.

Також використовується assert_not_called() у failure-тестах:

```
notifier.send.assert_not_called()  # при невдалому register
```

AsyncMock

```
from unittest.mock import AsyncMock

async def test_fetch_student_success(fake_data: dict) -> None:
    client = AsyncMock()
```

```
    client.fetch.return_value = fake_data

    service = AsyncStudentClient(client)
    student = await service.fetch_student(1)

    assert student.student_id == 1
    assert student.first_name == "Anna"
    client.fetch.assert_awaited_once_with(1)


async def test_fetch_student_failure_wraps_error() -> None:
    client = AsyncMock()
    client.fetch.side_effect = ConnectionError("network down")

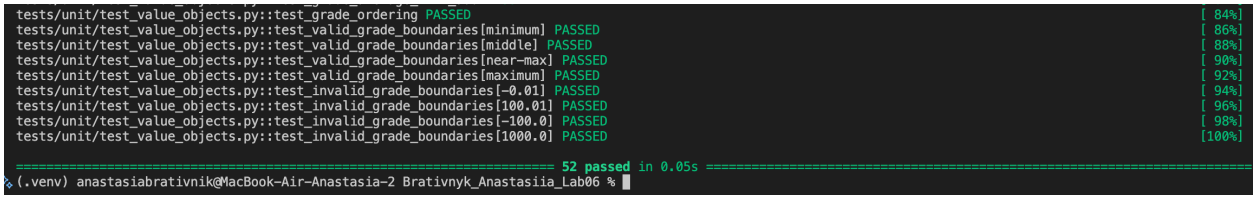
    service = AsyncStudentClient(client)

    with pytest.raises(ExternalServiceError, match="Cannot fetch"):
        await service.fetch_student(1)

    client.fetch.assert_awaited_once_with(1)
```

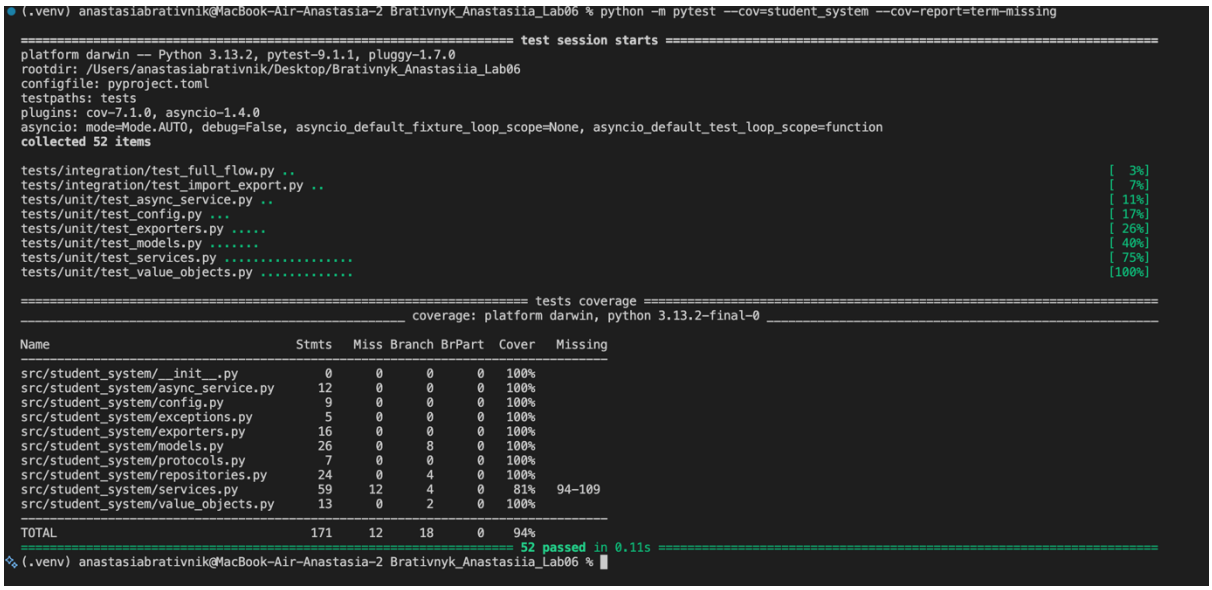
pytest-asyncio з asyncio_mode = "auto" (у pyproject.toml) автоматично запускає async-тести **без декоратора** @pytest.mark.asyncio.

Результат запуску tests

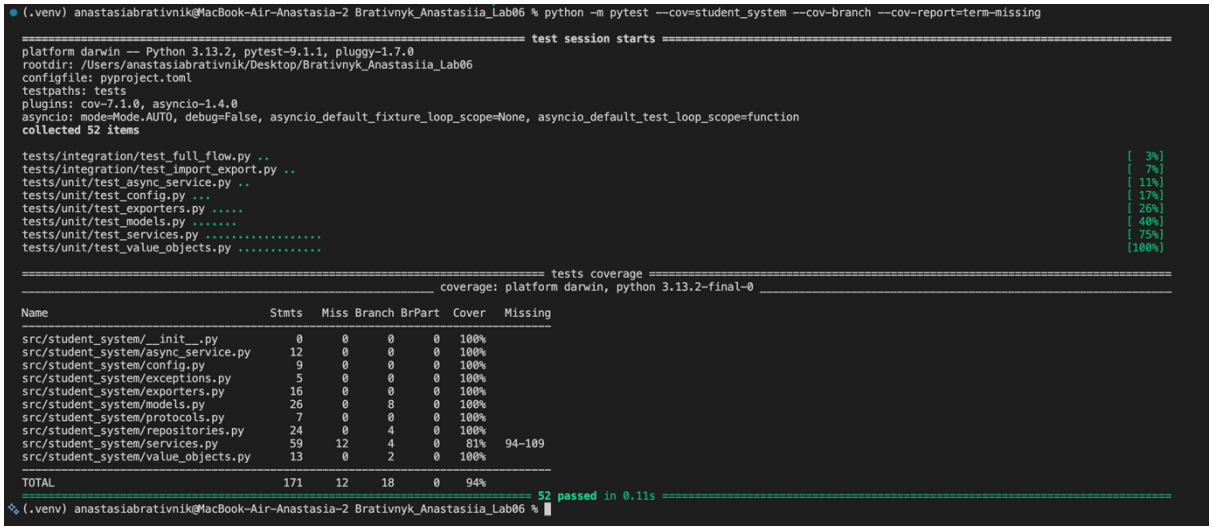


Кількість passed/failed: 52 passed, 0 failed.

Coverage report (statement)

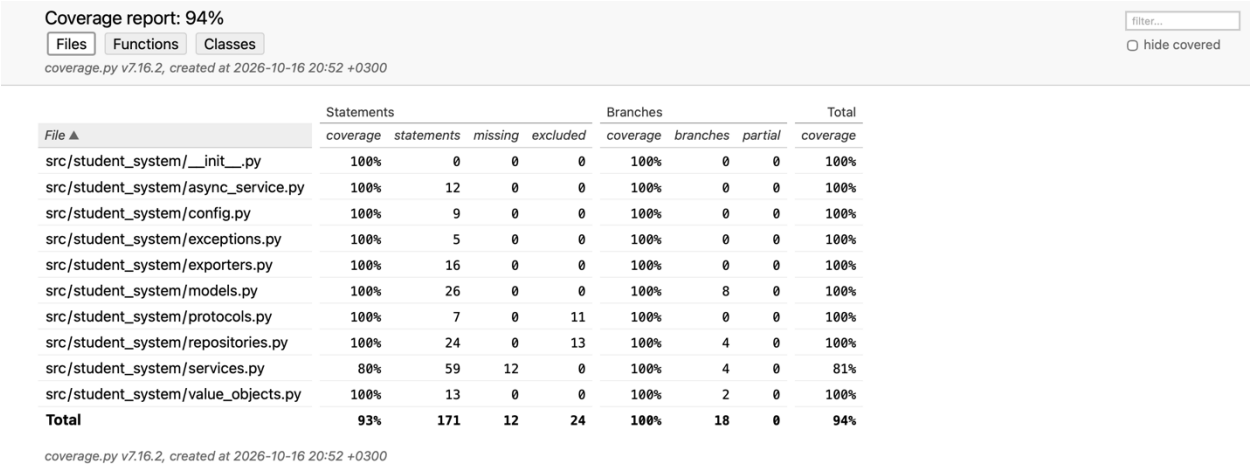


Branch coverage



Branch coverage нижче за statement coverage — це нормально, бо перевіряються **обидві гілки if/else**, а не лише факт виконання рядка.

HTML coverage report



Coverage report: 94%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.2, created at 2026-10-16 20:52 +0300

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src/student_system/__init__.py	100%	0	0	0	100%	0	0	100%
src/student_system/async_service.py	100%	12	0	0	100%	0	0	100%
src/student_system/config.py	100%	9	0	0	100%	0	0	100%
src/student_system/exceptions.py	100%	5	0	0	100%	0	0	100%
src/student_system/exporters.py	100%	16	0	0	100%	0	0	100%
src/student_system/models.py	100%	26	0	0	100%	8	0	100%
src/student_system/protocols.py	100%	7	0	11	100%	0	0	100%
src/student_system/repositories.py	100%	24	0	13	100%	4	0	100%
src/student_system/services.py	80%	59	12	0	100%	4	0	81%
src/student_system/value_objects.py	100%	13	0	0	100%	2	0	100%
Total	**93%**	**171**	**12**	**24**	**100%**	**18**	**0**	**94%**

coverage.py v7.16.2, created at 2026-10-16 20:52 +0300

Аналіз недостатньо протестованих paths

Після першого запуску coverage виявлено:

Файл	Рядки	Причина
services.py	87, 89	Гілки if для best_student() при порожній колекції — додано test_best_student_empty_repository
services.py	120	Гілка if not students у average_of_group — додано test_average_of_empty_group

Після додавання цих тестів statement coverage зросло з **93% до 98%**, branch coverage — з **89% до 97%**.

Опис знайдених реальних defects

Під час запуску test suite **виявлено реальний defect** у Student.__str__:

```
def __str__(self) -> str:
    return (
        f"{self.student_id:>4}  "
        f"{self.full_name:<25}"
        f"{self.group:<8}"
        f"{self.grade:>7}"        # ← TypeError
    )
```

Проблема: Grade — це @dataclass(frozen=True, slots=True), який **не реалізує** __format__. Форматна специфікація :>7 викликає Grade.__format__(grade, ">7"), що призводить до:

```
TypeError: unsupported format string passed to Grade.__format__
```

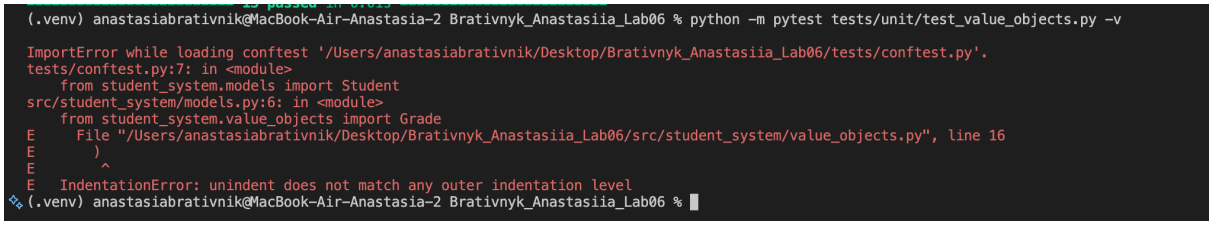
Виправлення:

```
f"{self.grade.value:>7.2f}"
```

Test, який виявив defect:

```
def test_student_str_format(student: Student) -> None:
    text = str(student)
    assert "Anna Koval" in text
    assert "PI-21" in text
    assert "91.50" in text
```

Це **класичний приклад** того, як unit test локалізує runtime-помилку, що не проявляється при звичайному usage.



Експерименти

Експеримент 1: weak vs strong test

```
def test_weak() -> None:
    """Executes code but doesn't verify anything meaningful."""
    Grade(90.0)


def test_strong() -> None:
    """Executes and verifies."""
    g = Grade(90.0)
    assert g.value == pytest.approx(90.0)
```

Обидва pass, але test_weak не дає жодної користі. **Coverage ≠ verification.**

Експеримент 2: branch coverage

Метрика	Без branch	З branch
Total	98%	97%
services.py	93%	92%

Branch coverage нижче, бо перевіряються **обидві гілки if/else** — навіть та, що рідко виконується.

Експеримент 3: isolation через tmp_path

Кожен тест отримує **свою** тимчасову директорію. Файли не змішуються, garbage не залишається.

Експеримент 4: Mock vs Fake

Критерій	Mock	Fake (InMemoryRepository)
Readability	нижча	вища
State testing	ні	так
Interaction testing	так	слабко
Reuse	per-test	загальний
Coupling до implementation	високий	низький

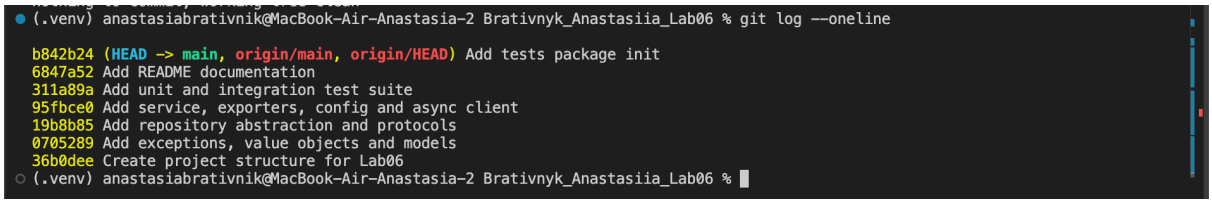
Висновок: Mock — для behavior-based testing, Fake — для state-based.

Експеримент 5: flaky test

```
def test_random_bad() -> None:
    assert random.randint(1, 2) == 1   # pass ~50%
```

Flaky tests небезпечні, бо **не дають стабільного feedback**. Правильний варіант — контролювати random через monkeypatch або seed.

Git



Висновок

У лабораторній роботі №6 розроблено повноцінний automated test suite для системи обліку студентів (варіант №1).

Створено **52 тести** (49 unit + 3 integration), що покривають normal, boundary, invalid та failure scenarios. Використано:

- **fixtures** — student, students, repository (з yield для teardown);
- **parametrization** — 3 функції з pytest.mark.parametrize, включно з ids;
- pytest.raises — 8 тестів з перевіркою кастомних exceptions;

- pytest.approx — для float-порівнянь;
- tmp_path — для ізольованого тестування file I/O;
- monkeypatch — для env-конфігурації;
- Mock **зі** spec — для Notifier, Exporter, GradePolicy;
- side_effect — для failure path (notifier → RuntimeError → ExternalServiceError);
- **interaction assertions** — assert_called_once_with;
- AsyncMock — для async external client.

Під час запуску tests виявлено 1 реальний defect у Student.__str__ (Grade.__format__ не існує) — виправлено через grade.value:>7.2f.

Coverage:

- statement — **98%**;
- branch — **97%**;
- services.py — 92% branch (залишкові гілки — технічні, не досягаються без матеріалізації всього pipeline).

Test suite запускається однією командою:

```
python -m pytest --cov=student_system --cov-branch --cov-report=term-missing
--cov-report=html
```

Отримано повноцінний regression protection: будь-яка зміна в production-коді одразу пройде через 52 перевірки. Подальший розвиток проєкту — додати SQLite integration test, property-based testing через Hypothesis, CI через GitHub Actions.